Technology Comparison

Project: DATERA EDU. Development of an intelligent system for monitoring students' knowledge acquisition using speech recognition and large language model technologies for analysing teacher-student dialogue

Team:

- David Khamitskiy (project manager, ML),
- Tairkhan Yermek (backend),
- Yermakhan Khamitov (frontend).

Supervisor: Khaimuldin Nursultan Gazizovich.

FR stands for a functional requirement and NFR for a non-functional one. The numbers after them are the numbers of the project requirements.

The work has three stages. The MVP is built from November 2026 to early March 2027: the teacher uploads materials and a recording, and the system gives the analysis, three reports and the knowledge map. All models in the MVP are ready-made. In late winter 2027 fine-tuning on open datasets is tried. Stage 2 runs from March to April 2027 and adds tests on weak topics, weekly summaries, notifications and fine-tuning on the teachers' corrections.

How the options were compared

For every decision three realistic options were taken and scored in the same way:

1. Every criterion comes from a project requirement, and its code is next to it.
2. A criterion has a weight. 3 is critical (without it a requirement cannot be met), 2 is important, 1 is nice to have.
3. Each option gets a score from 1 (poor) to 5 (very good) for every criterion.
4. The weighted total is the sum of score × weight. Each table shows the maximum.
5. If an option breaks a critical requirement, it is dropped, even with a good total.

The scores are the team's own assessment based on the documentation of each tool. Some decisions depend on numbers that are not yet available, such as recognition errors on real lessons. They will be measured, and the Verification Plan describes how.

Summary of decisions

#	Decision	Options compared	Chosen	Main reason
1	Backend platform	Go, Python, Node.js	Go	Fast API (NFR-1.3), small Docker images, the backend developer works in Go
2	Link between the API and the ML service	RabbitMQ, Redis Streams, direct call	RabbitMQ	Jobs are not lost and are retried; cabinets work when ML is down (NFR-2.1, NFR-2.3)

#	Decision	Options compared	Chosen	Main reason
3	Main database	PostgreSQL 16 + pgvector, MongoDB, PostgreSQL + Qdrant	PostgreSQL 16 + pgvector	Relational data and topic vectors in one database (FR-7.1, NFR-2.4)
4	File storage for audio and materials	MinIO, SeaweedFS, local disk	MinIO through the S3 API	Approved stack, signed links (NFR-3.5); easy to replace
5	Speech recognition model	faster-whisper large-v3, large-v3-turbo, cloud ASR	faster-whisper large-v3	One model for English, Russian and mixed speech (FR-6.1); audio stays on the project server (NFR-4.2)
6	LLM for materials, analysis and reports	Cloud LLM (Claude Sonnet class), cloud LLM (GPT class), local open model	MVP: cloud LLM with anonymization; stage 2: fine-tuned open model if it is good enough	Quality for NFR-5.5 and NFR-5.6; only anonymized text leaves the server (FR-7.6); adapter (FR-7.7)
7	Fine-tuning method	LoRA / QLoRA, full fine-tuning, prompts only	QLoRA	Fits one 24 GB GPU and 300 to 500 examples
8	Frontend framework	React + TypeScript, Vue 3, Angular	React + TypeScript	Team experience, libraries for the player and charts, types from OpenAPI (NFR-7.2)
9	Build tool	Vite, Next.js, Create React App	Vite	Login-only app, static files in Docker (NFR-7.1); CRA is no longer supported
10	UI components	shadcn/ui, Ant Design, MUI	shadcn/ui	Looks like the mockups, works from 360 px (NFR-6.2), small bundle (NFR-1.4)
11	Charts	Recharts, Apache ECharts, Chart.js	Recharts	React components, built-in lines for the 50 % and 80 % levels (FR-8.3)
12	Audio player	wavesurfer.js v7, HTML5 audio, Video.js or Plyr	wavesurfer.js v7	Waveform with markers and jumps to time codes (FR-9.4)
13	Data loading	TanStack Query, RTK Query, fetch + Context	TanStack Query	Status polling (FR-4.5), cache, retries, no global store

1. Backend platform (API server)

The API keeps accounts and roles, subjects and materials, uploads, the mastery index, reports, and a scheduler that deletes old audio. In stage 2 it also gets tests, summaries and notifications. The ML part is a separate Python service in any case, so only the language is chosen here for the business logic.

Criterion	Weight	Requirement
Performance and many requests at once	3	NFR-1.3 (95 % of requests under 500 ms), NFR-1.4 (up to 30 users at once)
Simple deployment in Docker	2	NFR-7.1, NFR-7.6 (stateless API)
Team experience	3	The backend developer works in Go
Libraries for the project tasks	2	JWT (NFR-3.3), RabbitMQ, S3, PostgreSQL, OpenAPI (NFR-7.2)
Type safety	1	NFR-7.4 (unit tests, 50 % coverage)

Criterion (weight)	Go (Gin or chi)	Python (FastAPI)	Node.js (NestJS)
Performance (3)	5	3	4
Deployment in Docker (2)	5	4	4
Team experience (3)	5	2	2
Libraries (2)	4	5	5
Type safety (1)	4	3	4
Weighted total (max 55)	**52**	**36**	**40**

Go is compiled, and goroutines let it serve many requests with little memory. The result is one static binary, so the Docker image is small. Go has fewer ready admin and ORM tools than Python, and some tasks take more code.

FastAPI is quick to write in and builds OpenAPI documents by itself. Python is also the language of the ML service, which sounds like a plus, but then the API and the ML code would start to mix. Python is also slower on CPU work.

NestJS would share TypeScript with the frontend and handles I/O-heavy APIs well. CPU tasks block its single event loop, though, and the backend developer does not mainly work with Node.js.

Go was chosen. It wins on response time (NFR-1.3), small Docker images (NFR-7.1) and team experience. The pilot load is small, 10 lessons a day and 30 users at once, so speed alone would not decide it. Team experience does: the backend developer writes Go, and this lowers the risk for the MVP in early March 2027. The choice between Gin and chi is small and is left for the first week of development. Both are thin layers over the Go standard library.

How it is checked: before the pilot, a load test of the main endpoints with 30 virtual users (NFR-1.3, NFR-1.4).

2. Link between the API and the ML service

A 60-minute lesson takes many minutes on a GPU: cleaning, recognition, diarization, then the LLM. The user should not wait for that, and a job must survive a service restart. The ML service gets three kinds of jobs. It splits new materials into topics (FR-3.5), processes a recording, and writes the reports after the API has calculated the index.

Criterion	Weight	Requirement
No lost jobs after a restart	3	NFR-2.1
Automatic retries and an error status	3	NFR-2.3 (up to 3 retries, then “Error”)

Criterion	Weight	Requirement
Cabinets work when ML is down	3	Recordings wait in the queue until ML is back
Many uploads processed in order	2	NFR-1.2 (at least 10 recordings a day, in a queue)
Simple infrastructure	1	Small team, one server
More ML workers when needed	2	NFR-7.6

Criterion (weight)	RabbitMQ	Redis Streams	Direct call (REST or gRPC)
No lost jobs (3)	5	3	1
Retries and error status (3)	5	3	2
Cabinets work when ML is down (3)	5	5	1
Uploads in order (2)	5	4	1
Simple infrastructure (1)	3	5	5
More ML workers (2)	5	4	2
Weighted total (max 70)	**68**	**54**	**23**

RabbitMQ keeps durable queues and persistent messages, so a restart loses nothing. A job leaves the queue only after the worker confirms it (manual ack). Failed jobs go to a dead-letter queue, several ML workers can read one queue, and Go and Python both have mature clients. The cost is one more service to run and watch.

Redis is already used for sessions and the login limit, so Redis Streams would add no new service. Its durability, however, depends on the AOF settings. There is no dead-letter queue, retries would have to be written as project code, and cache data and jobs would sit in one place.

A direct REST or gRPC call is the easiest to build and the worst for this system. The request would hang for up to 30 minutes and hit timeouts. If the ML service is down, the upload fails, and a restart loses the job. This breaks NFR-2.1 and the rule that cabinets keep working when ML is down.

RabbitMQ was chosen, because only a durable queue covers all three reliability points at once. Jobs survive restarts (NFR-2.1). A failed job is retried and then gets the “Error” status (NFR-2.3). The cabinets work while ML is down. To go on from the last finished stage (NFR-2.2), the API stores the stage of each lesson in PostgreSQL, and the message carries only the lesson ID and the job type.

How it is checked: stop the ML service in the middle of processing, start it again, and check that the lesson goes on from the last finished stage.

3. Main database

The data is relational. There are organizations, users and roles, links between student, teacher and subject, parent links, consents, materials and topics, lessons, the index history and the teacher’s status changes (FR-1 to FR-3, FR-8.6, FR-8.7). Topics also need vector search. The LLM looks for the places in a lesson where a confirmed topic came up (FR-7.1), and every rating

points to a part of the materials (FR-7.2). In stage 2, test questions are linked to topics as well (FR-10.3).

Criterion	Weight	Requirement
Fits relational data and access rules	3	FR-1.4, NFR-3.4 (rights checked on the server)
Transactions and consistent data	3	Consent records (FR-2.2), index history and status changes (FR-8.6, FR-8.7)
Vector search for topics and materials	2	FR-7.1, FR-7.2; stage 2: FR-10.3
One backup for all data	2	NFR-2.4 (daily backup, kept 7 days)
Simple infrastructure	1	Small team, one server

Criterion (weight)	PostgreSQL 16 + pgvector	MongoDB	PostgreSQL + Qdrant
Relational data and access (3)	5	2	5
Transactions (3)	5	3	5
Vector search (2)	4	3	5
One backup (2)	5	4	2
Simple infrastructure (1)	5	4	2
Weighted total (max 55)	**53**	**33**	**46**

PostgreSQL has foreign keys and ACID transactions, and a JSONB column can hold the JSON answer of the LLM. pgvector puts the vectors of topics and material fragments into the same database, with HNSW or IVFFlat indexes. A special vector database is faster at millions of vectors, but there will be thousands.

MongoDB stores flexible documents that look like the LLM output. The many-to-many links between student, teacher, subject and parent would need manual references, and the role checks would need more code.

Qdrant next to PostgreSQL would give the best vector search. It would also be a second database to run, back up and keep in sync, for a few thousand topics.

PostgreSQL 16 with pgvector was chosen. Strict access rules and transactions are what PostgreSQL does best. With so few topics and fragments, pgvector is fast enough, and it keeps one database, one backup (NFR-2.4) and simple joins between topics, materials and students.

How it is checked: real topic names are taken from the test lessons, matched with their topics and material fragments, and the share of correct matches is counted (FR-7.1, FR-7.2).

4. File storage for audio and materials

A lesson file can be up to 2 GB and 90 minutes long (FR-4.1). The video is deleted as soon as the audio is taken out of it (FR-4.4), and the audio is deleted after 30 days (FR-4.7, NFR-4.3). Audio is personal data, often of children. It has to stay on servers in Kazakhstan, be encrypted, and open only through a signed link that lives 15 minutes (NFR-3.5, NFR-3.6, NFR-4.1). Teachers' materials, up to 20 MB per file, are visible only inside their organization and also stay on the project server (FR-3.4, FR-3.7). MinIO is in the approved stack, but it has a known

problem: MinIO Community Edition stopped publishing binaries and Docker images in October 2025, and its repository was archived in 2026 [1][2].

Criterion	Weight	Requirement
Runs on the project server in Kazakhstan	3	NFR-4.1, NFR-4.2
Signed links, no direct access	3	NFR-3.5
Active maintenance and security updates	3	NFR-3.8, NFR-4
Standard S3 API, easy to replace	2	Services can be replaced through settings
Approved stack, team experience, documentation	2	MVP by early March 2027
Simple setup	1	Small team

Criterion (weight)	MinIO	SeaweedFS (S3 API)	Local disk
Own server in Kazakhstan (3)	5	5	5
Signed links (3)	5	4	4
Maintenance and updates (3)	1	5	4
S3 API (2)	5	4	1
Approved stack, experience (2)	5	2	3
Simple setup (1)	5	3	5
Weighted total (max 70)	**58**	**57**	**52**

The team already knows MinIO. It has the full S3 API with presigned links and starts in Docker in minutes. Its weak point is maintenance: since October 2025 there are no new official binaries or images, so the team has to follow security fixes itself.

SeaweedFS also runs on the project server. It has the Apache 2.0 licence, people still maintain it, and it speaks the S3 API. It is new to the team, it has more parts to set up, and signed links would have to be tested to confirm they work as needed.

A local disk is the simplest option. The Go API would hand out files through its own signed-link endpoint. Without an S3 API, though, any later move means code changes, and backups and encryption would be done by hand.

MinIO was chosen and is used only through the standard S3 API. Its total is only one point above SeaweedFS, so the risk is kept under control. One image version is pinned and latest is never used. MinIO is open only inside the Docker network, and users get files only through 15-minute links from the API. Audio is encrypted at rest with LUKS disk encryption or server-side encryption (NFR-3.6). Old audio is deleted by the backend scheduler after 30 days, not by MinIO. The code uses only the S3 API (AWS SDK for Go, boto3 for Python), so if a security problem shows up, the system can switch to SeaweedFS in the configuration.

How it is checked: a prototype in November 2026. A 2 GB file is uploaded, played through a 15-minute link, and the link is left to expire. It is also checked that MinIO is closed from outside and that the scheduler deletes the file.

5. Speech recognition model

Most MVP lessons are in English, often with explanations in Russian, or in Russian. The model must turn English, Russian and mixed speech into text with word time codes (FR-6.1) and work together with diarization. WER may be at most 15 % for English, 20 % for Russian and 25 % for mixed speech, and the last number is preliminary (NFR-5.1). A 60-minute lesson must be processed in 30 minutes (NFR-1.1).

Kazakh stays out of the MVP because it is recognized much worse. When people read texts aloud to Whisper fine-tuned by ISSAI, WER was 26.32 % for Kazakh, 11.86 % for Russian and 9.65 % for English [21]. No study of Kazakh lessons was found. WER is measured only on 3 to 5 Kazakh recordings.

Criterion	Weight	Requirement
Quality on English, Russian and mixed speech	3	NFR-5.1
One model for both languages and their mix	2	FR-6.1, FR-4.2 (“mixed / detect automatically”)
Speed on the project GPU	2	NFR-1.1
Audio does not leave the server	3	NFR-4.2
Word time codes	2	FR-6.1, FR-6.3, FR-6.6 (pauses)
Kazakh later	1	Future work

Criterion (weight)	faster-whisper large-v3	faster-whisper large-v3-turbo	Cloud ASR (Google, Yandex)
Quality (3)	5	4	4
One model (2)	5	4	3
Speed (2)	4	5	5
Audio stays on the server (3)	5	5	1
Word time codes (2)	5	5	4
Kazakh later (1)	4	4	3
Weighted total (max 65)	**62**	**59**	**42**

faster-whisper runs the open Whisper model on CTranslate2. It is faster and lighter than the original code and just as accurate [3]. One multilingual model reads English, Russian and their mix without switching, gives word time stamps and runs on the project’s own GPU. It needs enough GPU memory; 16 GB is planned for recognition, diarization and cleaning together. Whisper decides the language for a piece of audio, so inside a mixed phrase a word can come out in the wrong script. That is why mixed speech has its own WER target.

large-v3-turbo is the same tool with a much smaller decoder, 4 layers instead of 32, so it runs faster [4]. Accuracy drops a little, mostly on hard audio and mixed speech.

Cloud ASR from Google or Yandex handles English and Russian well and needs no project GPU. Usually one main language is set per request, so mixed speech suffers. Worse, lesson audio, often of children, would leave the project server and the country. That breaks NFR-4.1 and NFR-4.2, so this option is out.

faster-whisper large-v3 was chosen, with large-v3-turbo as a fallback. Quality has the highest weight, and mixed speech is the hardest case of the MVP. The language the teacher picks at upload (FR-4.2) goes to the model as a hint; with “mixed / detect automatically” the model finds the language itself. On long silence Whisper can write phrases nobody said [22], so voice activity detection skips such parts (FR-5.3). If a 60-minute lesson takes longer than 30 minutes (NFR-1.1), the system switches to turbo. The model sits behind one module, so adding Kazakh later means changing only the model.

How it is checked: on the test set (at least 20 lessons: 10 or more in English, 5 of them mixed, and 10 or more in Russian) WER is measured for each language and the processing time. On 3 to 5 Kazakh recordings a control WER is taken. If mixed speech turns out much worse, large-v3 and turbo are compared on it and such lessons are marked as lower accuracy.

6. LLM for materials, analysis and reports

The LLM does three jobs. It reads the teacher’s materials and proposes a list of topics, which the teacher then confirms (FR-3.5, FR-3.6). It reads the transcript of a lesson, about 8 to 12 thousand words, finds where each confirmed topic was discussed and checks the student’s answers against the materials. The answer is JSON with a status, reasons, time codes and links to the materials (FR-7.1, FR-7.2), and parts that are not in the materials get their own mark (FR-7.8). After that, the LLM writes the reports for the teacher, the student and the parent (FR-9.1). In stage 2 it can also make test questions from the materials (FR-10.2). The index, statuses and risk are always calculated by code.

The plan has two steps. The MVP uses a cloud LLM. In stage 2 an open model is fine-tuned on the project’s own data, and it replaces the cloud one through the same adapter, but only if it is good enough.

Criterion	**Weight**	**Requirement**
Quality on long lessons in English, Russian and mixed speech, with materials in the prompt	3	NFR-5.5 (status matches the teacher in at least 70 % of topics)
Valid JSON	2	NFR-5.6 (at least 95 % pass the schema on the first try)
Personal data protection	3	NFR-4.2, FR-7.6
Cost and setup time for the MVP	2	10 lessons a day, MVP in early March 2027
Can be fine-tuned and run on the project server later	2	FR-16.4, FR-16.6

Criterion (weight)	**Cloud LLM, Claude Sonnet class**	**Cloud LLM, GPT class**	**Local open model (Qwen or Llama, vLLM)**
Quality (3)	5	5	3
Valid JSON (2)	5	5	3
Personal data (3)	3	3	5
Cost and setup (2)	5	5	2
Fine-tuning later (2)	1	1	5

Criterion (weight)	Cloud LLM, Claude Sonnet class	Cloud LLM, GPT class	Local open model (Qwen or Llama, vLLM)
Weighted total (max 60)	**46**	**46**	**44**

Both cloud models handle long English and Russian texts well without any training. A whole lesson plus parts of the materials fits in their context, and no extra GPU is needed. The text does leave the project server, so names are replaced first (FR-7.6). Payment is per token, around tens of dollars a month at 10 lessons a day, and these models cannot be fine-tuned on the project server.

A local open model with 7 to 9 billion parameters keeps every word on the server and can be fine-tuned (decision 7). It needs a GPU with at least 24 GB. Without fine-tuning it is weaker, and fine-tuning needs data and setup time.

Published results suggest starting carefully. GPT-4o agreed with human coders of classroom talk on broad categories ($\kappa = 0.69$) but did poorly on fine-grained codes [23]. So the LLM is asked for three clear statuses instead of a fine scale, and every status has to point to a time code.

In the MVP a cloud LLM with anonymization is used, through the adapter. In stage 2 the fine-tuned open model is connected, if it is good enough. A cloud model reaches the quality needed for NFR-5.5 and NFR-5.6 without a second GPU and without training data. Privacy is covered because only anonymized text goes out (FR-7.6, NFR-4.2). The local model comes close in the total only thanks to fine-tuning and privacy, so it belongs to stage 2. The adapter (FR-7.7) turns the switch into a change of settings. The fine-tuned model goes in if its agreement with the teacher (NFR-5.5) is at most 5 percentage points below the cloud model's. Otherwise the cloud model stays, and the comparison becomes a result of the thesis. The two cloud options got the same score, so measurement will decide between them.

How it is checked: in the research stage 2 to 3 cloud models are compared on the control set of 5 to 10 lessons rated by a teacher (FR-16.2), using NFR-5.5 and NFR-5.6. The LLM is also given a hand-checked transcript and the recognized one of the same lessons, and the change in ratings caused by recognition errors is observed. In stage 2 the base, fine-tuned and cloud models are compared on the same control set (FR-16.5).

7. Fine-tuning method for the analysis model

The analysis model makes the main call of the system: how well a student understood each topic. Two fine-tuning runs are planned. The trial run, in late winter 2027, uses open teacher-student datasets, MathDial [9] and NCTE Transcripts [10] (FR-16.0). They describe what the teacher does and do not check answers, so the trial tests the process itself. The comparison task is fixed before training, for example the type of a teacher's turn. The main run, in early spring 2027, uses data from the pilot (FR-16.4). One example is a lesson fragment, a piece of the materials and the teacher's rating, confirmed or changed in the report (FR-8.7, FR-16.1). No extra labelling is needed. The target is at least 300 examples, ideally 500. The model is an open LLM with 7 to 9 billion parameters.

Criterion	Weight	Requirement
Fits one GPU with 24 GB	3	University server or rented GPU in Kazakhstan
Works with 300 to 500 examples	3	Size of the training set
Can raise agreement with the teacher	3	NFR-5.7 (better than the same model without fine-tuning)
Training time and cost	2	Early spring 2027, in parallel with the pilot
Approved stack	2	FR-16.4
Reproducible, small result files	1	NFR-7.7

Criterion (weight)	LoRA / QLoRA (Hugging Face PEFT)	Full fine-tuning	No fine-tuning (prompts and examples)
One 24 GB GPU (3)	5	1	5
300 to 500 examples (3)	5	2	5
Raises agreement (3)	4	5	2
Time and cost (2)	4	1	5
Approved stack (2)	5	1	2
Reproducible (1)	5	3	4
Weighted total (max 70)	**65**	**31**	**54**

LoRA trains only small adapter matrices and leaves the base model frozen [11]. QLoRA loads the base model in 4-bit form, so a model with 7 to 9 billion parameters trains on one 24 GB GPU [12]. A few hundred examples are enough to start. The result is a small adapter file that is easy to version, and vLLM can serve the base model with it. The ceiling is a bit lower than with full fine-tuning, and the rank and learning rate need tuning.

Full fine-tuning changes all weights and can reach the best quality when there is a lot of data. The project does not have that. A 7 to 9 billion model would need several large GPUs, 300 to 500 examples would overfit it, and every version would be a full copy of the model. It fails the critical GPU criterion, so it is dropped.

Prompts with examples need no training. They also leave the project without its own model, so NFR-5.7 and the research part of the thesis stay open, and long prompts slow down every request.

QLoRA with Hugging Face Transformers and PEFT was chosen. It is the only method that fits the available GPU and the small dataset (FR-16.4). The prompts-only option stays in the study as the baseline, “the same model without fine-tuning”, for NFR-5.7. One training script runs twice: first on MathDial and NCTE, then on the pilot examples split by lessons (FR-16.3). That way the process is tested before the real data is ready. Only lessons with a separate consent go into training (FR-2.3, NFR-4.5), and the data is deleted from a rented GPU after the run.

How it is checked: in the trial, the fine-tuned and the base model are compared on a held-out part of the open data (FR-16.0). In stage 2, the base, fine-tuned and cloud models are compared on the control set (FR-16.5, NFR-5.7), and the model version is saved and training settings are saved (NFR-7.7).

Frontend (decisions 8 to 13)

The web client has three cabinets: one for the organization (administrator and teacher), one for the student and one for the parent. Yermakhan Khamitov builds it. The Figma mockups cover login, the student list, upload of materials and a recording, the processing status, the teacher's report with the "Confirm" and "Change" buttons, the knowledge map, and the reports for the student and the parent. Desktop frames are 1440 × 1024, phone frames 402 × 874, and the layout must still work from 360 px (NFR-6.2).

The requirements below shape most frontend choices.

Requirement	What it means for the frontend
FR-3.4 to FR-3.7	Upload of materials (PDF, DOCX, PPTX, TXT, MD, up to 20 MB); the teacher confirms, renames, merges, deletes or adds topics
FR-4.1 to FR-4.6	Upload of a recording up to 2 GB with progress; status "Uploaded", "Audio cleaning", "Recognition", "Speaker separation", "Analysis", "Reports", then "Ready" or "Error"; restart after an error
FR-6.4, FR-6.7	Transcript with roles in color; the teacher fixes speaker labels, and the analysis runs again
FR-7.8, FR-8.7	Parts outside the materials are shown to the teacher; "Confirm" and "Change" on every topic status, with an optional comment
FR-8.2	Knowledge map: topics with color, index, date of the last lesson and the direction of change
FR-9.2 to FR-9.4	Report language from the profile; every item with a time code opens the player at that second, or the transcript after the audio is deleted
FR-11 to FR-13	Student list with progress, lesson feed, "What to repeat", child switch for parents
Stage 2	Index chart (FR-8.3), tests in the browser (FR-10.7), "Need attention" (FR-11.2), notifications (FR-14.1)
NFR-1.4	A cabinet page loads in 3 s or less, up to 30 users at once
NFR-3.3, NFR-3.8	JWT: 15-minute access token and a refresh token; protection against XSS and CSRF
NFR-6.1 to NFR-6.5	Two last versions of Chrome (Edge, Firefox, Safari are wanted); screens from 360 px; interface in Russian with texts in translation files; parent report without special terms; upload a recording and open a report in 3 steps or less
NFR-7.2, NFR-7.3	API described in OpenAPI; linters and tests on every change

8. Frontend framework

The interface has many pages with forms, tables, a player and charts, and one developer has to build it before early March 2027. So the framework needs ready libraries for a waveform player, charts and UI components.

Criterion	Weight	Requirement
Team experience	3	The frontend developer works with React
Libraries for the project features	3	FR-9.4 (player), FR-8.2, FR-8.3 (map, charts), UI components, OpenAPI client
Type safety and API types	2	NFR-7.2

Criterion	Weight	Requirement
Speed of work for one developer	2	MVP in early March 2027
Page load speed	1	NFR-1.4

Criterion (weight)	React + TypeScript	Vue 3 + TypeScript	Angular
Team experience (3)	5	2	1
Libraries (3)	5	4	3
Types (2)	5	4	5
Speed of work (2)	4	5	2
Page load (1)	4	5	3
Weighted total (max 55)	**52**	**41**	**29**

React has the largest set of libraries. wavesurfer.js, Recharts, shadcn/ui and TanStack Query all support it well, and TypeScript catches API mistakes during the build. React is only a view library, so routing, forms and data loading are separate choices (decisions 9 to 13).

Vue 3 is easy to learn, its templates look like plain HTML, and it comes with an official router and store. The frontend developer knows it less, and fewer component libraries look like the mockups.

Angular has routing, forms and an HTTP client in one package and a strict structure. For one developer with a short deadline it is too heavy: it is hard to learn and needs a lot of code even for small pages.

React with TypeScript was chosen. The waveform player, charts and accessible UI components already exist for it, so the developer’s time goes into pages instead of low-level widgets. TypeScript types generated from OpenAPI (NFR-7.2) keep the frontend in sync with the Go API.

How it is checked: a prototype in November 2026 with login, one list page and the player with a time-code jump, on a mock API.

9. Build tool

Every page is behind login (FR-1), so search engines never see them. The client is a set of static files that talks to the Go API over REST.

Criterion	Weight	Requirement
Active maintenance and security updates	3	NFR-3.8
Fits a login-only app (no SEO, no server rendering)	2	FR-1
Simple deployment as static files in Docker	2	NFR-7.1, NFR-7.6
Fast work (dev server, hot reload)	2	MVP in early March 2027
Page load time (code splitting, cache)	2	NFR-1.4

Criterion (weight)	Vite (single-page app)	Next.js (server rendering)	Create React App
Maintenance (3)	5	5	1
Login-only app (2)	5	3	4
Static files in Docker (2)	5	3	5
Fast work (2)	5	4	2
Page load (2)	4	5	3
Weighted total (max 55)	**53**	**45**	**31**

Vite starts its dev server almost at once. The production build is plain static files that nginx serves from a small container, with the code split by route. On the first visit the browser downloads the whole app's JavaScript, so the bundle size needs watching (NFR-1.4).

Next.js renders pages on the server and shows the first screen fast. In production it needs a Node.js server, which is one more service, and server rendering gives nothing for pages behind login.

The React team deprecated Create React App in February 2025 [13]. No new features and no security updates means it fails a critical criterion, and it is dropped.

Vite was chosen as a single-page application. Server rendering does not help cabinets behind login, and static files are the easiest thing to run in Docker next to the API (NFR-7.1). To stay under 3 seconds (NFR-1.4), the code is split by route, and each cabinet loads only its own pages.

How it is checked: Lighthouse on the production build with the network slowed to 10 Mbit/s (Verification Plan, section 4.4).

10. UI components

The interface has to look like the mockups (white background, blue accents, cards, tables, badges, the Inter font) and work on phones from 360 px.

Criterion	Weight	Requirement
Looks like the mockups	3	Approved Figma mockups
Layout from 360 px	3	NFR-6.2
Ready parts for the screens (tables, forms, dialogs, tabs)	2	FR-3.6, FR-8.7, FR-11.1
Small bundle	2	NFR-1.4
Accessibility (keyboard, screen readers)	1	Usability in the pilot (Verification Plan, C6)

Criterion (weight)	shadcn/ui (Radix UI + Tailwind CSS)	Ant Design	MUI
Looks like the mockups (3)	5	3	2
Layout from 360 px (3)	5	3	4
Ready parts (2)	4	5	5
Small bundle (2)	5	3	3
Accessibility (1)	5	3	4
Weighted total (max 55)	**53**	**37**	**38**

shadcn/ui copies components into the project, and Tailwind styles them, so they can match the mockups exactly. Radix UI underneath gives keyboard support and ARIA attributes. Only the parts that are used end up in the bundle, and Tailwind breakpoints make the 360 px layout easy. There is no ready data table, so sorting and paging come from TanStack Table, and the copied components are updated by hand.

Ant Design has almost everything: tables with sorting, forms, upload, and a Russian locale. It also has a strong look of its own, and matching the mockups would need a custom theme. Its big tables are hard to use on a phone, and the bundle is larger.

MUI is mature and accessible, with many components. Its Material Design look is far from the mockups.

shadcn/ui with Tailwind CSS was chosen. The design match and the phone layout carry the most weight (NFR-6.2), and shadcn/ui is clearly ahead on both. The colors, radius and font from the mockups go into the Tailwind theme once, and every component picks them up. Ant Design has more ready parts, but restyling it would eat the time those parts save.

How it is checked: the teacher home page and the student list are built from the mockups and compared at 1440 px and 360 px.

11. Charts

In the MVP the cabinets show the index of each topic on the knowledge map (FR-8.2, FR-12.2, FR-13.3) and the average index in the student list (FR-11.1). Stage 2 adds a line chart of the index of a topic and of a subject over time, with lines at 50 % and 80 % (FR-8.3). Each chart has tens of points, no more.

Criterion	**Weight**	**Requirement**
Needed types: lines with level lines, bars, tooltips	3	FR-8.2, FR-8.3
Charts as React components	2	Speed of work
Resize down to 360 px	2	NFR-6.2
Bundle size	2	NFR-1.4

Criterion (weight)	**Recharts**	**Apache ECharts**	**Chart.js (react-chartjs-2)**
Needed types (3)	5	5	4
React components (2)	5	3	3
Resize to 360 px (2)	4	4	4
Bundle size (2)	4	3	4
Weighted total (max 45)	**41**	**35**	**34**

In Recharts a chart is a React component, and reference lines for 50 % and 80 % are built in. It draws SVG, which stays sharp and is easy to color in the project palette. SVG slows down with thousands of points, and there will never be that many.

ECharts is very powerful and draws large data on canvas. It is set up through one big options object instead of components, and the full package is heavy.

Chart.js is small and popular. Level lines need an extra annotation plugin, and in React it works through a wrapper.

Recharts was chosen. The charts are small and simple, so the power of ECharts would go unused, while Recharts fits React best and has the level lines out of the box. Most charts arrive only in stage 2 (FR-8.3), so if the pilot shows a problem, the switch will be cheap.

How it is checked: the knowledge map is built from the mockups with real data from the test set.

12. Audio player with time codes

Reports contain time codes of hard moments (FR-7.2, FR-9.3), and a click has to open the player at that second (FR-9.4). Students see the player in their lesson feed (FR-12.1). Recordings last 60 to 90 minutes (FR-4.1). The video is deleted right away (FR-4.4), so the player works with audio only, through 15-minute signed links (NFR-3.5). After 30 days the audio is gone, and a time code opens the transcript instead (FR-4.7, FR-9.4).

Criterion	Weight	Requirement
Jump to a time code from any report item	3	FR-9.4
Long files with low memory, streamed by a signed link	3	FR-4.1, NFR-3.5
Waveform with markers of hard moments	2	FR-9.3, FR-9.4, mockups
Effort to build	2	MVP in early March 2027

Criterion (weight)	wavesurfer.js v7	HTML5 audio + own timeline	Video.js or Plyr
Jump to a time code (3)	5	5	5
Long files, low memory (3)	4	5	5
Waveform with markers (2)	5	1	2
Effort (2)	4	3	4
Weighted total (max 50)	**45**	**38**	**42**

wavesurfer.js draws the waveform, has regions and markers as plugins, seeks by time and is written in TypeScript. If the browser decodes a long file by itself, a 60-minute track can eat more than 500 MB of memory [14]. This is avoided by streaming through the HTML media element and drawing from peaks computed in advance.

Plain HTML5 audio streams files of any length with little memory and needs no library. It has no waveform, though, and the markers and the timeline would have to be written by the team.

Video.js and Plyr are polished players with good streaming. They are built for video, which are not kept, they have no waveform, and markers need extra plugins.

wavesurfer.js v7 with the media element and computed peaks was chosen. It is the only option with the waveform and markers that the mockups and FR-9.4 ask for. The memory problem is solved the way its documentation suggests [14]: the audio streams through an <audio> element by the signed link, and the waveform comes from a small array of about 1,000 to 2,000 peaks. The server would compute the peaks once at the preparation stage and send them with the lesson;

this still needs to be agreed with the backend. Two edge cases are handled in the player. When the 15-minute link expires, the client asks for a new one and continues from the same second. When the audio has been deleted, a time code opens the transcript fragment. Time codes on every rating may also help with trust: teachers trusted AI labels of classroom talk more when they saw explanations [24].

How it is checked: with a 90-minute file, the memory of the browser tab, the time until the waveform appears, the accuracy of a jump (± 1 s) and the behaviour after the link expires are observed.

13. Data loading and state

Almost everything in the cabinets comes from the API: lists, reports, topics, the knowledge map. The processing status changes on the server (FR-4.5), and the page has to show it without a reload. Some teacher actions change data on other pages: fixed speaker labels (FR-6.4), a changed topic status (FR-8.7), an edited topic list (FR-3.6). The access token lives only 15 minutes (NFR-3.3).

Criterion	**Weight**	**Requirement**
Polling of the processing status	3	FR-4.5, FR-4.6
Cache and refresh after changes	2	FR-3.6, FR-6.4, FR-8.7; stage 2: FR-10.6
Simple code for one developer	2	MVP in early March 2027
Types generated from OpenAPI	2	NFR-7.2
Errors, retries and token refresh	2	NFR-3.3

Criterion (weight)	**TanStack Query**	**Redux Toolkit + RTK Query**	**fetch in useEffect + Context**
Status polling (3)	5	5	2
Cache and refresh (2)	5	5	2
Simple code (2)	4	2	4
OpenAPI types (2)	4	5	3
Errors and tokens (2)	4	4	2
Weighted total (max 55)	**49**	**47**	**28**

TanStack Query puts cache, loading and error states, retries and polling (refetchInterval) into one hook. Polling can stop on its own when the status becomes “Ready” or “Error”. It works with a typed client from openapi-typescript and openapi-fetch. Query keys and cache invalidation take a little time to learn.

RTK Query can do the same and has an official generator from OpenAPI. It comes with a Redux store, and there is nothing to put there: almost all the state is server data.

fetch inside useEffect needs no library at all. Then every page would need its own cache, polling, retries and protection from race conditions, written by hand, and that is where bugs come from.

TanStack Query with a typed client from openapi-typescript and openapi-fetch was chosen. It and RTK Query are almost equal, and TanStack Query wins because it has no global store and needs less code. The little local state, like the current child of a parent or the filters, lives in the URL or in React state. After “Confirm” or “Change”, the client invalidates the report and the

knowledge map so they show the new index. Token refresh sits in one place, the API client middleware: on a 401 answer it calls the refresh endpoint once and repeats the request. Tokens never go into localStorage. The access token stays in memory, and the refresh token is an httpOnly cookie set by the API, which keeps it away from XSS (NFR-3.8). This part still needs agreement with the backend.

How it is checked: a file is uploaded and the status page is watched; a new stage must appear without a reload within 10 s. Then the page is left open for more than 15 minutes and it is checked that it still works without a new login.

Other frontend tools

These are small, standard choices, so they are not scored.

Task	Tool	Requirement
Routing and pages by role	React Router	FR-1.4 (the API still checks rights, NFR-3.4)
Forms and validation	react-hook-form + zod	FR-1, FR-3.4, FR-4.2
Interface texts in translation files	react-i18next (Russian now, other languages later)	NFR-6.3
Typed API client	openapi-typescript + openapi-fetch	NFR-7.2
Tables with sorting and paging	TanStack Table	FR-11.1
Mock API before the backend is ready	Mock Service Worker (MSW)	Frontend and backend work in parallel
Unit and component tests	Vitest + React Testing Library	NFR-7.3
End-to-end tests	Playwright	Verification Plan, section 7
Code style	ESLint + Prettier in CI	NFR-7.3
Serving the build	nginx container in Docker Compose	NFR-7.1

File upload works the same way for recordings and materials (FR-3.4, FR-4.1 to FR-4.3). The browser asks the API for a signed upload link and sends the file straight to MinIO in one PUT request. S3 accepts up to 5 GB in one request, and the project limit is 2 GB. Progress comes from XMLHttpRequest upload events, because fetch does not report upload progress in every browser. Before the upload starts, the client checks the format, the size and the student's consent (FR-2.1). Resuming a broken upload (FR-4.8) is future work. MinIO has to accept requests from the web client's domain (CORS), and that is a backend setting.

Agreement with the requirements

All 13 decisions meet the project requirements. Decisions 8 to 13 match the approved frontend stack: React with TypeScript, Vite, shadcn/ui, Recharts, TanStack Query and wavesurfer.js. Decisions 5 to 7 follow the MVP languages (English, Russian and mixed speech) and the two-step plan for the analysis model. One risk remains open:

Item	Risk	How it is controlled
File storage (MinIO)	MinIO Community Edition is no longer maintained (no binaries since October 2025, repository archived in 2026)	Fixed image version, no access from outside the Docker network, encryption at rest, only the S3 API in the code; SeaweedFS is the replacement

Every final decision will also go into the repository as an Architecture Decision Record.

Open questions

- Gin or chi for the Go API. Both are fine, and one is picked in the first week of development.
- The cleaning model: DeepFilterNet, Demucs or RNNoise. The research stage decides by WER before and after cleaning (FR-5.2, NFR-5.2).
- A tool to pull text out of PDF, DOCX and PPTX materials (FR-3.5). Scans without a text layer are not supported in the MVP (FR-3.4), so a standard text extractor should do.
- The exact cloud LLM, chosen in the research stage by NFR-5.5 and NFR-5.6.
- The open model for fine-tuning (Qwen or Llama family, 7 to 9 billion parameters) and the licences of MathDial and NCTE, to check before the trial run in late winter 2027.
- Frontend and backend: where tokens are stored (proposed: the access token in memory and the refresh token in an httpOnly cookie) and how the refresh request is protected from CSRF (NFR-3.3, NFR-3.8).
- Frontend and backend: waveform peaks computed at the preparation stage and returned with the lesson (decision 12).
- Frontend and backend: the status is polled every 5 s in the MVP. Server-sent events can replace polling later, and the pages will not change.
- Nobody owns the screens outside the three cabinets yet: the system administrator’s queue and prompt pages (FR-15.2, FR-15.3) and the page where a teacher rates the control lessons (FR-16.2). Both are part of the MVP.

References

1. Glukhov, R. MinIO CE in 2026: Retired Upstream, Source-Only, and What to Use. https://glukhov.org/data-infrastructure/object-storage/minio-dead/
2. Pinggy. MinIO Just Got Archived. Here’s What Self-Hosters Are Running Instead. https://pinggy.io/blog/minio_archived_self_hosted_s3_alternatives/
3. SYSTRAN. faster-whisper (README, benchmarks). https://github.com/SYSTRAN/faster-whisper
4. OpenAI. whisper-large-v3-turbo (model card). https://huggingface.co/openai/whisper-large-v3-turbo
5. pgvector: open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector
6. RabbitMQ documentation. Reliability guide. https://www.rabbitmq.com/docs/reliability
7. SeaweedFS. https://github.com/seaweedfs/seaweedfs
8. pyannote.audio. https://github.com/pyannote/pyannote-audio

9. J. Macina et al. MathDial: A Dialogue Tutoring Dataset with Rich Pedagogical Properties Grounded in Math Reasoning Problems. Findings of EMNLP 2023. https://aclanthology.org/2023.findings-emnlp.372/
10. D. Demszky, H. Hill. The NCTE Transcripts: A Dataset of Elementary Math Classroom Transcripts. BEA 2023. https://aclanthology.org/2023.bea-1.44
11. E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. https://arxiv.org/abs/2106.09685
12. T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. https://arxiv.org/abs/2305.14314
13. React team. Sunsetting Create React App (14 February 2025). https://react.dev/blog/2025/02/14/sunsetting-create-react-app
14. wavesurfer.js documentation. Performance and memory. https://wavesurfer.xyz/docs/performance/
15. Vite. https://vite.dev/
16. shadcn/ui. https://ui.shadcn.com/
17. Ant Design. https://ant.design/
18. Recharts. https://recharts.org/
19. Apache ECharts. https://echarts.apache.org/
20. TanStack Query. https://tanstack.com/query
21. Z. Makhataeva, N. Atymtay, R. Meiramov, G. Nauryzbaikyzy, K. Sadirova, H. A. Varol. Multilingual automatic speech recognition interface for typing: Usability study and performance evaluation for Kazakh, Russian, and English. Scientific Journal of Astana IT University, vol. 24, pp. 49-63, 2025. https://doi.org/10.37943/24AHNP6638
22. A. Koenecke, A. S. G. Choi, K. X. Mei, H. Schellmann, M. Sloane. Careless Whisper: Speech-to-text hallucination harms. ACM FAccT 2024. https://doi.org/10.1145/3630106.3658996
23. S. Xu, X. Huang, C. K. Lo, G. Chen, M. S.-Y. Jong. Evaluating the performance of ChatGPT and GPT-4o in coding classroom discourse data. Computers and Education: Artificial Intelligence, vol. 7, 2024, 100325. https://doi.org/10.1016/j.caeai.2024.100325
24. D. Wang, C. Bian, G. Chen. Using explainable AI to unravel classroom dialogue analysis: Effects of explanations on teachers' trust, technology acceptance and cognitive load. British Journal of Educational Technology, vol. 55, no. 6, pp. 2530-2556, 2024. https://doi.org/10.1111/bjet.13466